Task 1: Process Management and Threading

Design Documentation — ST5004CEM Operating Systems and Security

Overview

This program (task1_threads.cpp) demonstrates process/thread creation, synchronization, CPU scheduling, and two of the most common concurrency hazards: race conditions and deadlocks. It is deliberately kept to a single file so that every requirement can be traced to one short, readable piece of code.

Threading Model

Four worker threads are created (exceeding the minimum of three requested in the brief) and each is given two jobs that require sharing a limited resource: printing a document, and parking in one of two spaces. Threads are created with std::thread and joined with .join() so the main thread waits for all work to finish before the program exits.

Synchronization Mechanisms

Two different synchronization primitives are used, matching different real-world situations:

- A std::mutex (printerMutex) protects the shared printer. Only one thread may hold it at a time, modelling a resource that can only ever be used by one process at once (e.g. a single physical device).
- A std::counting_semaphore<10> (parkingSpots, initialised to 2) models a resource with a fixed number of interchangeable units. Up to two threads may hold a permit at once; a third thread calling acquire() blocks until a permit is released.

Both use RAII-style locking (std::lock_guard, and acquire()/release() called explicitly) so that a resource is never left locked if an exception were thrown.

Round-Robin Scheduler Simulation

A std::queue<SimpleProcess> models the ready queue. Each process is given a fixed quantum (2 time units). On each turn, the process at the front of the queue runs for min(quantum, remainingTime); if it still has time left, it is pushed to the back of the queue, otherwise it is marked finished. With three processes (burst times 5, 3 and 7), the simulation output shows each process being interleaved fairly rather than run to completion one at a time, which is the defining behaviour of round-robin scheduling.

Race Condition Demonstration

The same increment loop is run on four threads in two ways. incrementUnsafe() reads the shared counter into a local variable, calls std::this_thread::yield() (deliberately widening the window in which another

thread can interleave), and then writes back local+1. incrementSafe() performs the same operation but inside a std::lock_guard on a dedicated mutex.

With 4 threads each incrementing 5,000 times, the expected total is 20,000. In testing, the unsafe counter consistently finished at 5,000 each thread's updates overwrote the others' because the read-modify-write sequence was not atomic. The safe counter finished at exactly 20,000 on every run. This is the race condition made visible and then corrected: the mutex guarantees only one thread can execute the read-modify-write sequence at a time, so no update is ever lost.

Deadlock and Its Prevention

A deadlock can occur when two threads each hold one lock and are waiting for the other (e.g. Thread A holds lockA and waits for lockB, while Thread B holds lockB and waits for lockA). This program avoids that scenario entirely by using std::lock(lockA, lockB), which acquires both mutexes together as a single atomic operation. The C++ standard library guarantees this function cannot deadlock regardless of the order in which different threads call it, because it uses a deadlock-avoidance algorithm internally (trying locks and backing off) rather than a fixed acquisition order. Two threads calling safePairOfLocks() concurrently were observed to both complete successfully with no hang.

Design Decisions

All primitives used are from the C++20 standard library (std::thread, std::mutex, std::counting_semaphore, std::lock) rather than a third-party threading library, to keep the dependency list minimal and the mechanisms transparent. The program is organised into four clearly labelled sections so each requirement can be located, read, and explained independently rather than being spread across an entangled codebase.